Synchronous FIFO USING SV

Design code

```
// DUT of synchronous FIFO
module Sync_fifo #(parameter DATA_WIDTH=8,
        parameter FIFO_DEPTH=16,
        parameter ADDR_WIDTH=4)(

        input logic clk,
        input logic rst_n,
        input logic rd_en,
        input logic wr_en,
        input logic [DATA_WIDTH-1:0] data_in,
        output logic [DATA_WIDTH-1:0] data_out,
      output logic underflow,
      output logic overflow,
      output logic full,
      output logic empty
);

logic [DATA_WIDTH-1:0] memory [FIFO_DEPTH];
logic [ADDR_WIDTH-1:0] wr_ptr, rd_ptr;
logic [ADDR_WIDTH:0] count;

always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
                wr_ptr   <= '0;
                rd_ptr   <= '0;
```

```
        count    <= '0;
        data_out <= '0;
        overflow <= 1'b0;
        underflow<= 1'b0;
    end
    else begin
            overflow  <= 1'b0;
            underflow <= 1'b0;

            if (wr_en && !full) begin
                    memory[wr_ptr] <= data_in;
                    wr_ptr <= (wr_ptr == FIFO_DEPTH-1) ? '0 : wr_ptr + 1'b1;
            end
            else if (wr_en && full) begin
                    overflow <= 1'b1;
            end

            if (rd_en && !empty) begin
                    data_out <= memory[rd_ptr];
                    rd_ptr <= (rd_ptr == FIFO_DEPTH-1) ? '0 : rd_ptr + 1'b1;
            end
            else if (rd_en && empty) begin
                    underflow <= 1'b1;
            end

            case ({(wr_en && !full), (rd_en && !empty)})
                    2'b10: count <= count + 1'b1;
```

```
                2'b01: count <= count - 1'b1;
                2'b11: count <= count;
                default: count <= count;
            endcase
        end
end

always_comb begin
        full  = (count == FIFO_DEPTH);
        empty = (count == 0);
end

endmodule
```

Interface: -

```
// interface of FIFO
interface fifo_if #(parameter DATA_WIDTH = 8) (input logic clk);
        logic rst_n;
        logic wr_en;
        logic rd_en;
        logic [DATA_WIDTH-1:0] data_in;
        logic [DATA_WIDTH-1:0] data_out;
        logic full;
        logic empty;
        logic overflow;
        logic underflow;
endinterface
```

// Packet /Transaction

```
class fifo_packet;
        rand bit wr_en;
      rand bit rd_en;
      rand bit [7:0] data_in;
        bit [7:0] data_out;
        bit full;
      bit empty;
      bit overflow;
      bit underflow;


        function void display(string class_name);
                $display("[class name=%s] wr_en=%0b rd_en=%0b
data_in=%0d",class_name,wr_en,rd_en,data_in);
        endfunction
endclass
```

// Generator of FIFO

```
class generator;
        mailbox #(fifo_packet) gen2drv;


        function new(mailbox #(fifo_packet) mbx);
                this.gen2drv = mbx;
        endfunction


        task run();
                fifo_packet pkt;
```

```
        $display("[GEN] : WRITE PHASE HAS BEEN STARTED");
        repeat (20) begin
            pkt = new();
            pkt.randomize() with { wr_en==1; rd_en==0; };
            gen2drv.put(pkt);
            pkt.display("GEN");
        end

        $display("[GEN] : READ PHASE HAS BEEN STARTED");
        repeat (20) begin
            pkt = new();
            pkt.randomize() with { wr_en==0; rd_en==1; };
            gen2drv.put(pkt);
            pkt.display("GEN");
        end

        $display("[GEN] : READ-WRITE PHASE HAS BEEN STARTED");
        repeat (10) begin
            pkt = new();
            pkt.randomize() with { wr_en==1; rd_en==1; };
            gen2drv.put(pkt);
            pkt.display("GEN");
        end
    endtask
endclass
```

// Driver for FIFO

```
class driver;
        mailbox #(fifo_packet) gen2drv;
        virtual fifo_if vif;

        function new(mailbox #(fifo_packet) mbx, virtual fifo_if vif);
                this.gen2drv = mbx;
                this.vif     = vif;
        endfunction

        task reset();
                vif.rst_n <= 0;
                vif.wr_en <= 0;
                vif.rd_en <= 0;
                vif.data_in <= 0;
                repeat(5) @(posedge vif.clk);
                vif.rst_n <= 1;
                $display("[DRV] DUT RESET HAS BEEN DONE succesfully!!!");
        endtask

        task run();
                fifo_packet pkt;
                reset();

                forever begin
                        gen2drv.get(pkt);
                        @(posedge vif.clk);
```

```
                vif.wr_en   <= pkt.wr_en;
                vif.rd_en   <= pkt.rd_en;
                vif.data_in <= pkt.data_in;
                @(posedge vif.clk);
                vif.wr_en <= 0;
                vif.rd_en <= 0;

                if (pkt.wr_en && !pkt.rd_en)
                        $display("[DRV] WRITE data = %0d", pkt.data_in);
                else if (pkt.rd_en && !pkt.wr_en)
                        $display("[DRV] READ request");
                else if (pkt.wr_en && pkt.rd_en)
                        $display("[DRV] WRITE-READ data = %0d", pkt.data_in);
        end
    endtask
endclass
```

//Monitor of FIFO

```
class monitor;
    virtual fifo_if vif;
    mailbox #(fifo_packet) mon2sb;
    fifo_coverage cov;

    function new(mailbox #(fifo_packet) mbx, virtual fifo_if vif);
        this.mon2sb = mbx;
        this.vif    = vif;
```

```
        cov = new(vif);
endfunction

task run();
        fifo_packet pkt;

        $display("START Printing the Monitor DATA");
        forever begin
                @(posedge vif.clk);
                pkt = new();
                pkt.wr_en     = vif.wr_en;
                pkt.rd_en     = vif.rd_en;
                pkt.data_in   = vif.data_in;
                pkt.full      = vif.full;
                pkt.empty     = vif.empty;
                pkt.overflow  = vif.overflow;
                pkt.underflow = vif.underflow;


                if (vif.rd_en && !vif.empty) begin
                        @(posedge vif.clk);
                        pkt.data_out = vif.data_out;
                end
                else begin
                        pkt.data_out = 'x;
                end
```

```
                mon2sb.put(pkt);
                $display("[MON] wr=%0b rd=%0b din=%0d dout=%0d full=%0b empty=%0b ovf=%0b udf=%0b",pkt.wr_en, pkt.rd_en, pkt.data_in, pkt.data_out,pkt.full, pkt.empty, pkt.overflow, pkt.underflow);
            end
        endtask
endclass
```

// Scoreboardof FIFO

```
class scoreboard;
    mailbox #(fifo_packet) mon2sb;
    byte ref_q[$];

    function new(mailbox #(fifo_packet) mbx);
        this.mon2sb = mbx;
    endfunction



    task run();
        fifo_packet pkt;
        byte exp_data;

        forever begin
            mon2sb.get(pkt);

            if (pkt.rd_en && !pkt.empty) begin
                if (ref_q.size() == 0) begin
```

```
                $error("[%0t] SCB ERROR: Read when reference FIFO
empty", $time);
            end
            else begin
                exp_data = ref_q.pop_front();
                if (pkt.data_out !== exp_data) begin
                    $error("[%0t] SCB MISMATCH: Exp=%0d
Got=%0d",$time, exp_data, pkt.data_out);
                end
                else begin
                    $display("[SCB] DATA READ OK : %0d",
pkt.data_out);
                end
            end
        end

        if (pkt.wr_en && !pkt.full) begin
            ref_q.push_back(pkt.data_in);
            $display("[SCB] DATA WRITE OK : %0d", pkt.data_in);
        end



        if (pkt.overflow)
            $display("[%0t] SCB: OVERFLOW detected", $time);
        if (pkt.underflow)
            $display("[%0t] SCB: UNDERFLOW detected", $time);
    end
endtask
```

endclass

// Testbench of FIFO

```
`include "design.sv"
`include "interface.sv"
`include "packet.sv"
`include "generator.sv"
`include "driver.sv"
`include "coverage.sv"
`include "monitor.sv"
`include "scoreboard.sv"

module tb;
parameter DATA_WIDTH = 8;
parameter FIFO_DEPTH = 16;
parameter ADDR_WIDTH = 4;

logic clk;

initial clk = 0;
always #5 clk = ~clk;

fifo_if #(DATA_WIDTH) vif (clk);

Sync_fifo #(.DATA_WIDTH(8),.FIFO_DEPTH(16),.ADDR_WIDTH(4)) dut
(.clk(clk),.rst_n(vif.rst_n),.wr_en(vif.wr_en),.rd_en(vif.rd_en),.data_in(vif.data_in),.data
_out (vif.data_out),.full(vif.full),.empty(vif.empty),.overflow(vif.overflow),.underflow
(vif.underflow));
```

```
mailbox #(fifo_packet) gen2drv;
mailbox #(fifo_packet) mon2sb;

generator  gen;
driver     drv;
monitor    mon;
scoreboard sb;

initial begin
        gen2drv = new();
        mon2sb  = new();

        gen = new(gen2drv);
      drv = new(gen2drv, vif);
      mon = new(mon2sb,  vif);
      sb  = new(mon2sb);

        fork
                gen.run();
                drv.run();
            mon.run();
            sb.run();
        join_none

        #1150;
        $display("ALL Test has been succesfully completed!!!");
```

```
        $finish;
end
initial begin
        $fsdbDumpfile("fifo.fsdb");
        $fsdbDumpvars(0,tb);
end
endmodule
```

// Coverage of FIFO

```
class fifo_coverage;
        virtual fifo_if vif;

        covergroup fifo_cg @(posedge vif.clk);
                cp_wr_en : coverpoint vif.wr_en {
                        bins write = {1};
                       bins no_write = {0};
                }

              cp_rd_en : coverpoint vif.rd_en {
                        bins read = {1};
                       bins no_read = {0};
                   }

             cp_op : coverpoint {vif.wr_en, vif.rd_en} {
                        bins WRITE_ONLY = {2'b10};
                    bins READ_ONLY  = {2'b01};
                    bins READ_WRITE = {2'b11};
```

```
        bins IDLE       = {2'b00};
      }


    cp_full  : coverpoint vif.full {
        bins full = {1};
        bins not_full = {0};
      }

    cp_empty : coverpoint vif.empty {
        bins empty = {1};
        bins not_empty = {0};
      }


    cp_overflow : coverpoint vif.overflow {
          bins ovf = {1};
     }

    cp_underflow : coverpoint vif.underflow {
          bins udf = {1};
     }


    cross_wr_full : cross cp_wr_en, cp_full {
          bins write_when_full = binsof(cp_wr_en.write) &&
binsof(cp_full.full);
   }
```

```
        cross_rd_empty : cross cp_rd_en, cp_empty {
                bins read_when_empty = binsof(cp_rd_en.read) &&
binsof(cp_empty.empty);
    }

        cross_op_state : cross cp_op, cp_full, cp_empty;
    endgroup

    function new(virtual fifo_if vif);
        this.vif = vif;
        fifo_cg = new();
    endfunction
endclass
```

Run command

```
[b23n_sarif_727@prac3 lab20]$ vcs -sverilog top.sv -kdb -lca -debug_access+all -full64
./simv
verdi -cov -covdir simv.vdb/
```

File name: -

```
[b23n_sarif_727@prac3 lab20]$ ls
coverage.sv  dump.fsdb     inter.fsdb      novas.rc       simv.daidir  verdiLog
csrc         fifo.fsdb     monitor.sv      packet.sv      simv.vdb
design.sv    generator.sv  novas.conf      scoreboard.sv  top.sv
driver.sv    interface.sv  novas_dump.log  simv           ucli.key
[b23n_sarif_727@prac3 lab20]$ verdi -cov -covdir simv.vdb/
```

Simulation Result

Output

[GEN] : WRITE PHASE HAS BEEN STARTED

[class name=GEN] wr_en=1 rd_en=0 data_in=220

[class name=GEN] wr_en=1 rd_en=0 data_in=217

[class name=GEN] wr_en=1 rd_en=0 data_in=204

[class name=GEN] wr_en=1 rd_en=0 data_in=25

[class name=GEN] wr_en=1 rd_en=0 data_in=47

[class name=GEN] wr_en=1 rd_en=0 data_in=201

[class name=GEN] wr_en=1 rd_en=0 data_in=31

[class name=GEN] wr_en=1 rd_en=0 data_in=248

[class name=GEN] wr_en=1 rd_en=0 data_in=21

[class name=GEN] wr_en=1 rd_en=0 data_in=27

[class name=GEN] wr_en=1 rd_en=0 data_in=216

[class name=GEN] wr_en=1 rd_en=0 data_in=195

[class name=GEN] wr_en=1 rd_en=0 data_in=125

[class name=GEN] wr_en=1 rd_en=0 data_in=109

[class name=GEN] wr_en=1 rd_en=0 data_in=118

[class name=GEN] wr_en=1 rd_en=0 data_in=47

[class name=GEN] wr_en=1 rd_en=0 data_in=113

[class name=GEN] wr_en=1 rd_en=0 data_in=28

[class name=GEN] wr_en=1 rd_en=0 data_in=25

[class name=GEN] wr_en=1 rd_en=0 data_in=155

[GEN] : READ PHASE HAS BEEN STARTED

[class name=GEN] wr_en=0 rd_en=1 data_in=72

[class name=GEN] wr_en=0 rd_en=1 data_in=18

[class name=GEN] wr_en=0 rd_en=1 data_in=82

[class name=GEN] wr_en=0 rd_en=1 data_in=161

[class name=GEN] wr_en=0 rd_en=1 data_in=184

[class name=GEN] wr_en=0 rd_en=1 data_in=134

[class name=GEN] wr_en=0 rd_en=1 data_in=211

[class name=GEN] wr_en=0 rd_en=1 data_in=64

[class name=GEN] wr_en=0 rd_en=1 data_in=73

[class name=GEN] wr_en=0 rd_en=1 data_in=194

[class name=GEN] wr_en=0 rd_en=1 data_in=171

[class name=GEN] wr_en=0 rd_en=1 data_in=28

[class name=GEN] wr_en=0 rd_en=1 data_in=29

[class name=GEN] wr_en=0 rd_en=1 data_in=83

[class name=GEN] wr_en=0 rd_en=1 data_in=97

[class name=GEN] wr_en=0 rd_en=1 data_in=230

[class name=GEN] wr_en=0 rd_en=1 data_in=225

[class name=GEN] wr_en=0 rd_en=1 data_in=34

[class name=GEN] wr_en=0 rd_en=1 data_in=98

[class name=GEN] wr_en=0 rd_en=1 data_in=2

[GEN] : READ-WRITE PHASE HAS BEEN STARTED

[class name=GEN] wr_en=1 rd_en=1 data_in=226

[class name=GEN] wr_en=1 rd_en=1 data_in=38

[class name=GEN] wr_en=1 rd_en=1 data_in=214

[class name=GEN] wr_en=1 rd_en=1 data_in=161

[class name=GEN] wr_en=1 rd_en=1 data_in=217

[class name=GEN] wr_en=1 rd_en=1 data_in=245

[class name=GEN] wr_en=1 rd_en=1 data_in=198

[class name=GEN] wr_en=1 rd_en=1 data_in=54

[class name=GEN] wr_en=1 rd_en=1 data_in=240

[class name=GEN] wr_en=1 rd_en=1 data_in=87

START Printing the Monitor DATA

[MON] wr=0 rd=0 din=0 dout=0 full=0 empty=1 ovf=0 udf=0

[MON] wr=0 rd=0 din=0 dout=0 full=0 empty=1 ovf=0 udf=0

[MON] wr=0 rd=0 din=0 dout=0 full=0 empty=1 ovf=0 udf=0

[MON] wr=0 rd=0 din=0 dout=0 full=0 empty=1 ovf=0 udf=0

[DRV] DUT RESET HAS BEEN DONE succesfully!!!

[MON] wr=0 rd=0 din=0 dout=0 full=0 empty=1 ovf=0 udf=0

[MON] wr=0 rd=0 din=0 dout=0 full=0 empty=1 ovf=0 udf=0

[DRV] WRITE data = 220

[MON] wr=1 rd=0 din=220 dout=0 full=0 empty=1 ovf=0 udf=0

[SCB] DATA WRITE OK : 220

[MON] wr=0 rd=0 din=220 dout=0 full=0 empty=0 ovf=0 udf=0

[DRV] WRITE data = 217

[MON] wr=1 rd=0 din=217 dout=0 full=0 empty=0 ovf=0 udf=0

[SCB] DATA WRITE OK : 217

[MON] wr=0 rd=0 din=217 dout=0 full=0 empty=0 ovf=0 udf=0

[DRV] WRITE data = 204

[MON] wr=1 rd=0 din=204 dout=0 full=0 empty=0 ovf=0 udf=0

[SCB] DATA WRITE OK : 204

[MON] wr=0 rd=0 din=204 dout=0 full=0 empty=0 ovf=0 udf=0

[DRV] WRITE data = 25

[MON] wr=1 rd=0 din=25 dout=0 full=0 empty=0 ovf=0 udf=0

[SCB] DATA WRITE OK : 25

[MON] wr=0 rd=0 din=25 dout=0 full=0 empty=0 ovf=0 udf=0

[DRV] WRITE data = 47

[MON] wr=1 rd=0 din=47 dout=0 full=0 empty=0 ovf=0 udf=0

[SCB] DATA WRITE OK : 47

[MON] wr=0 rd=0 din=47 dout=0 full=0 empty=0 ovf=0 udf=0

[DRV] WRITE data = 201

[MON] wr=1 rd=0 din=201 dout=0 full=0 empty=0 ovf=0 udf=0

[SCB] DATA WRITE OK : 201

[MON] wr=0 rd=0 din=201 dout=0 full=0 empty=0 ovf=0 udf=0

[DRV] WRITE data = 31

[MON] wr=1 rd=0 din=31 dout=0 full=0 empty=0 ovf=0 udf=0

[SCB] DATA WRITE OK : 31

[MON] wr=0 rd=0 din=31 dout=0 full=0 empty=0 ovf=0 udf=0

[DRV] WRITE data = 248

[MON] wr=1 rd=0 din=248 dout=0 full=0 empty=0 ovf=0 udf=0

[SCB] DATA WRITE OK : 248

[MON] wr=0 rd=0 din=248 dout=0 full=0 empty=0 ovf=0 udf=0

[DRV] WRITE data = 21

[MON] wr=1 rd=0 din=21 dout=0 full=0 empty=0 ovf=0 udf=0

[SCB] DATA WRITE OK : 21

[MON] wr=0 rd=0 din=21 dout=0 full=0 empty=0 ovf=0 udf=0

[DRV] WRITE data = 27

[MON] wr=1 rd=0 din=27 dout=0 full=0 empty=0 ovf=0 udf=0

[SCB] DATA WRITE OK : 27

[MON] wr=0 rd=0 din=27 dout=0 full=0 empty=0 ovf=0 udf=0

[DRV] WRITE data = 216

[MON] wr=1 rd=0 din=216 dout=0 full=0 empty=0 ovf=0 udf=0

[SCB] DATA WRITE OK : 216

[MON] wr=0 rd=0 din=216 dout=0 full=0 empty=0 ovf=0 udf=0

[DRV] WRITE data = 195

[MON] wr=1 rd=0 din=195 dout=0 full=0 empty=0 ovf=0 udf=0

[SCB] DATA WRITE OK : 195

[MON] wr=0 rd=0 din=195 dout=0 full=0 empty=0 ovf=0 udf=0

[DRV] WRITE data = 125

[MON] wr=1 rd=0 din=125 dout=0 full=0 empty=0 ovf=0 udf=0

[SCB] DATA WRITE OK : 125

[MON] wr=0 rd=0 din=125 dout=0 full=0 empty=0 ovf=0 udf=0

[DRV] WRITE data = 109

[MON] wr=1 rd=0 din=109 dout=0 full=0 empty=0 ovf=0 udf=0

[SCB] DATA WRITE OK : 109

[MON] wr=0 rd=0 din=109 dout=0 full=0 empty=0 ovf=0 udf=0

[DRV] WRITE data = 118

[MON] wr=1 rd=0 din=118 dout=0 full=0 empty=0 ovf=0 udf=0

[SCB] DATA WRITE OK : 118

[MON] wr=0 rd=0 din=118 dout=0 full=0 empty=0 ovf=0 udf=0

[DRV] WRITE data = 47

[MON] wr=1 rd=0 din=47 dout=0 full=0 empty=0 ovf=0 udf=0

[SCB] DATA WRITE OK : 47

[MON] wr=0 rd=0 din=47 dout=0 full=1 empty=0 ovf=0 udf=0

[DRV] WRITE data = 113

[MON] wr=1 rd=0 din=113 dout=0 full=1 empty=0 ovf=0 udf=0

[MON] wr=0 rd=0 din=113 dout=0 full=1 empty=0 ovf=1 udf=0

[395] SCB: OVERFLOW detected

[DRV] WRITE data = 28

[MON] wr=1 rd=0 din=28 dout=0 full=1 empty=0 ovf=0 udf=0

[MON] wr=0 rd=0 din=28 dout=0 full=1 empty=0 ovf=1 udf=0

[415] SCB: OVERFLOW detected

[DRV] WRITE data = 25

[MON] wr=1 rd=0 din=25 dout=0 full=1 empty=0 ovf=0 udf=0

[MON] wr=0 rd=0 din=25 dout=0 full=1 empty=0 ovf=1 udf=0

[435] SCB: OVERFLOW detected

[DRV] WRITE data = 155

[MON] wr=1 rd=0 din=155 dout=0 full=1 empty=0 ovf=0 udf=0

[MON] wr=0 rd=0 din=155 dout=0 full=1 empty=0 ovf=1 udf=0

[455] SCB: OVERFLOW detected

[DRV] READ request

[MON] wr=0 rd=1 din=72 dout=220 full=1 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 220

[DRV] READ request

[MON] wr=0 rd=1 din=18 dout=217 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 217

[DRV] READ request

[MON] wr=0 rd=1 din=82 dout=204 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 204

[DRV] READ request

[MON] wr=0 rd=1 din=161 dout=25 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 25

[DRV] READ request

[MON] wr=0 rd=1 din=184 dout=47 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 47

[DRV] READ request

[MON] wr=0 rd=1 din=134 dout=201 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 201

[DRV] READ request

[MON] wr=0 rd=1 din=211 dout=31 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 31

[DRV] READ request

[MON] wr=0 rd=1 din=64 dout=248 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 248

[DRV] READ request

[MON] wr=0 rd=1 din=73 dout=21 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 21

[DRV] READ request

[MON] wr=0 rd=1 din=194 dout=27 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 27

[DRV] READ request

[MON] wr=0 rd=1 din=171 dout=216 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 216

[DRV] READ request

[MON] wr=0 rd=1 din=28 dout=195 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 195

[DRV] READ request

[MON] wr=0 rd=1 din=29 dout=125 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 125

[DRV] READ request

[MON] wr=0 rd=1 din=83 dout=109 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 109

[DRV] READ request

[MON] wr=0 rd=1 din=97 dout=118 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 118

[DRV] READ request

[MON] wr=0 rd=1 din=230 dout=47 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 47

[DRV] READ request

[MON] wr=0 rd=1 din=225 dout=0 full=0 empty=1 ovf=0 udf=0

[MON] wr=0 rd=0 din=225 dout=0 full=0 empty=1 ovf=0 udf=1

[795] SCB: UNDERFLOW detected

[DRV] READ request

[MON] wr=0 rd=1 din=34 dout=0 full=0 empty=1 ovf=0 udf=0

[MON] wr=0 rd=0 din=34 dout=0 full=0 empty=1 ovf=0 udf=1

[815] SCB: UNDERFLOW detected

[DRV] READ request

[MON] wr=0 rd=1 din=98 dout=0 full=0 empty=1 ovf=0 udf=0

[MON] wr=0 rd=0 din=98 dout=0 full=0 empty=1 ovf=0 udf=1

[835] SCB: UNDERFLOW detected

[DRV] READ request

[MON] wr=0 rd=1 din=2 dout=0 full=0 empty=1 ovf=0 udf=0

[MON] wr=0 rd=0 din=2 dout=0 full=0 empty=1 ovf=0 udf=1

[855] SCB: UNDERFLOW detected

[DRV] WRITE-READ data = 226

[MON] wr=1 rd=1 din=226 dout=0 full=0 empty=1 ovf=0 udf=0

[SCB] DATA WRITE OK : 226

[MON] wr=0 rd=0 din=226 dout=0 full=0 empty=0 ovf=0 udf=1

[875] SCB: UNDERFLOW detected

[DRV] WRITE-READ data = 38

[MON] wr=1 rd=1 din=38 dout=226 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 226

[SCB] DATA WRITE OK : 38

[DRV] WRITE-READ data = 214

[MON] wr=1 rd=1 din=214 dout=38 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 38

[SCB] DATA WRITE OK : 214

[DRV] WRITE-READ data = 161

[MON] wr=1 rd=1 din=161 dout=214 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 214

[SCB] DATA WRITE OK : 161

[DRV] WRITE-READ data = 217

[MON] wr=1 rd=1 din=217 dout=161 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 161

[SCB] DATA WRITE OK : 217

[DRV] WRITE-READ data = 245

[MON] wr=1 rd=1 din=245 dout=217 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 217

[SCB] DATA WRITE OK : 245

[DRV] WRITE-READ data = 198

[MON] wr=1 rd=1 din=198 dout=245 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 245

[SCB] DATA WRITE OK : 198

[DRV] WRITE-READ data = 54

[MON] wr=1 rd=1 din=54 dout=198 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 198

[SCB] DATA WRITE OK : 54

[DRV] WRITE-READ data = 240

[MON] wr=1 rd=1 din=240 dout=54 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 54

[SCB] DATA WRITE OK : 240

[DRV] WRITE-READ data = 87

[MON] wr=1 rd=1 din=87 dout=240 full=0 empty=0 ovf=0 udf=0

[SCB] DATA READ OK : 240

[SCB] DATA WRITE OK : 87

[MON] wr=0 rd=0 din=87 dout=0 full=0 empty=0 ovf=0 udf=0

[MON] wr=0 rd=0 din=87 dout=0 full=0 empty=0 ovf=0 udf=0

[MON] wr=0 rd=0 din=87 dout=0 full=0 empty=0 ovf=0 udf=0

[MON] wr=0 rd=0 din=87 dout=0 full=0 empty=0 ovf=0 udf=0

[MON] wr=0 rd=0 din=87 dout=0 full=0 empty=0 ovf=0 udf=0

[MON] wr=0 rd=0 din=87 dout=0 full=0 empty=0 ovf=0 udf=0

[MON] wr=0 rd=0 din=87 dout=0 full=0 empty=0 ovf=0 udf=0

[MON] wr=0 rd=0 din=87 dout=0 full=0 empty=0 ovf=0 udf=0

[MON] wr=0 rd=0 din=87 dout=0 full=0 empty=0 ovf=0 udf=0

ALL Test has been succesfully completed!!!

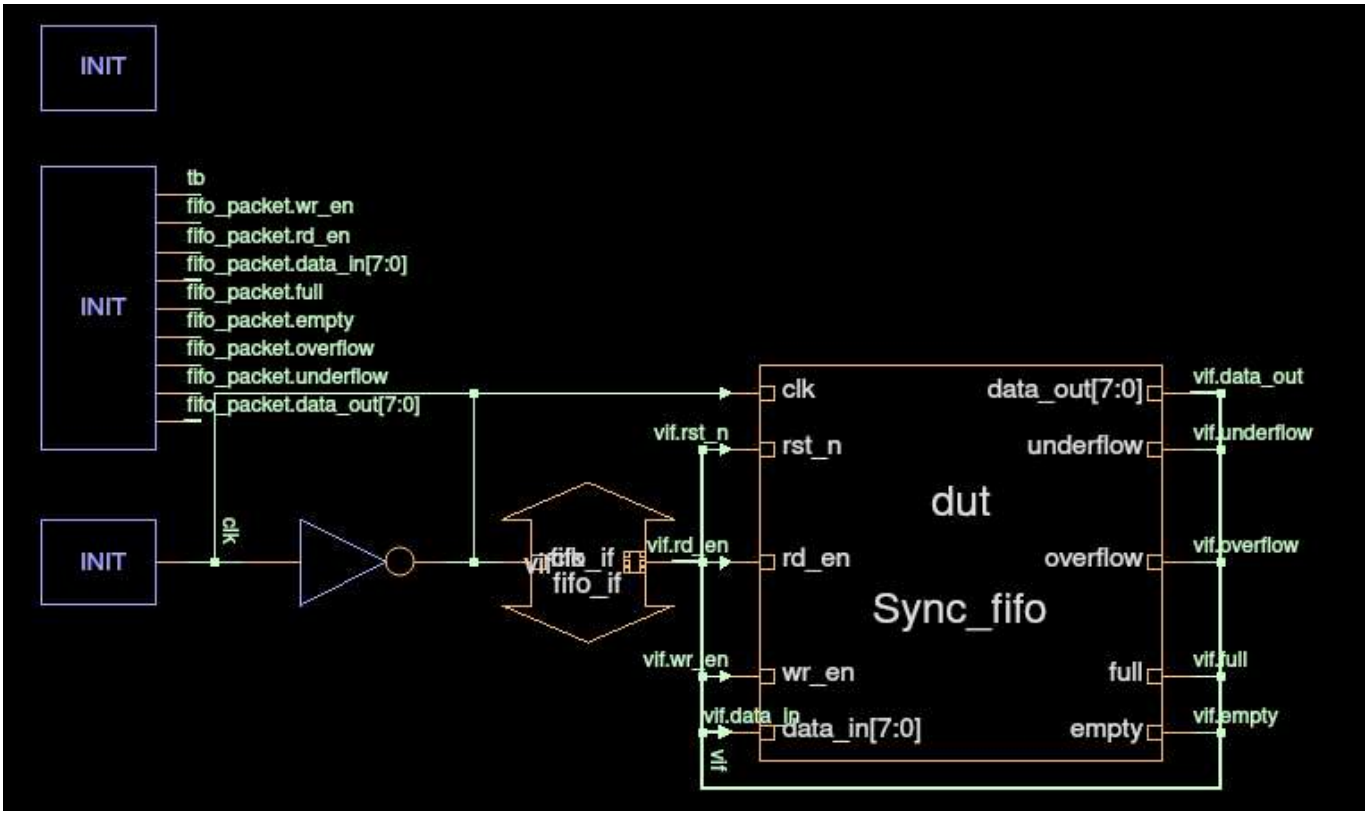
$finish called from file "top.sv", line 52.

$finish at simulation time 1150

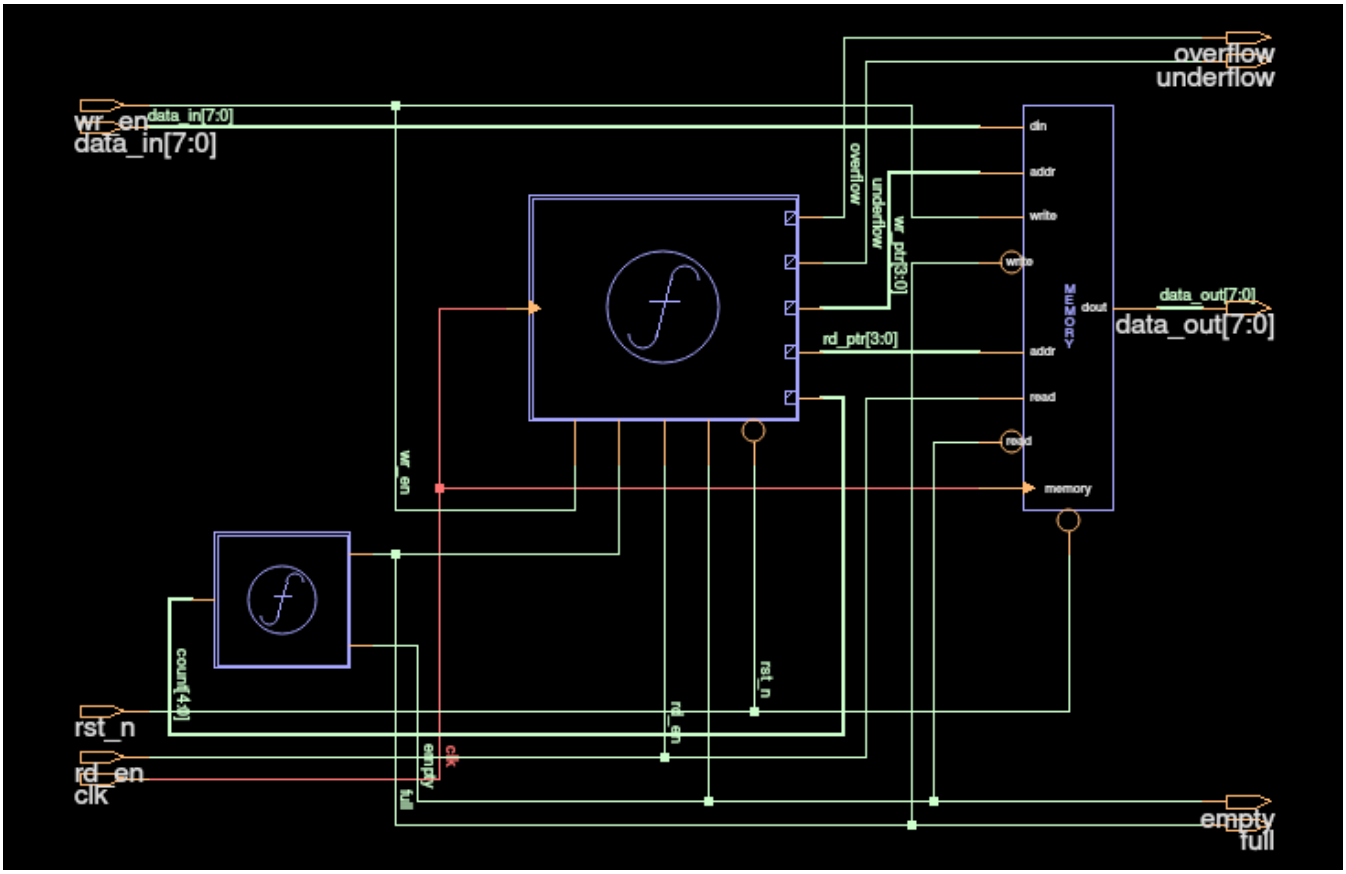
V C S S i m u l a t i o n R e p o r t

Time: 1150

Schematic Design



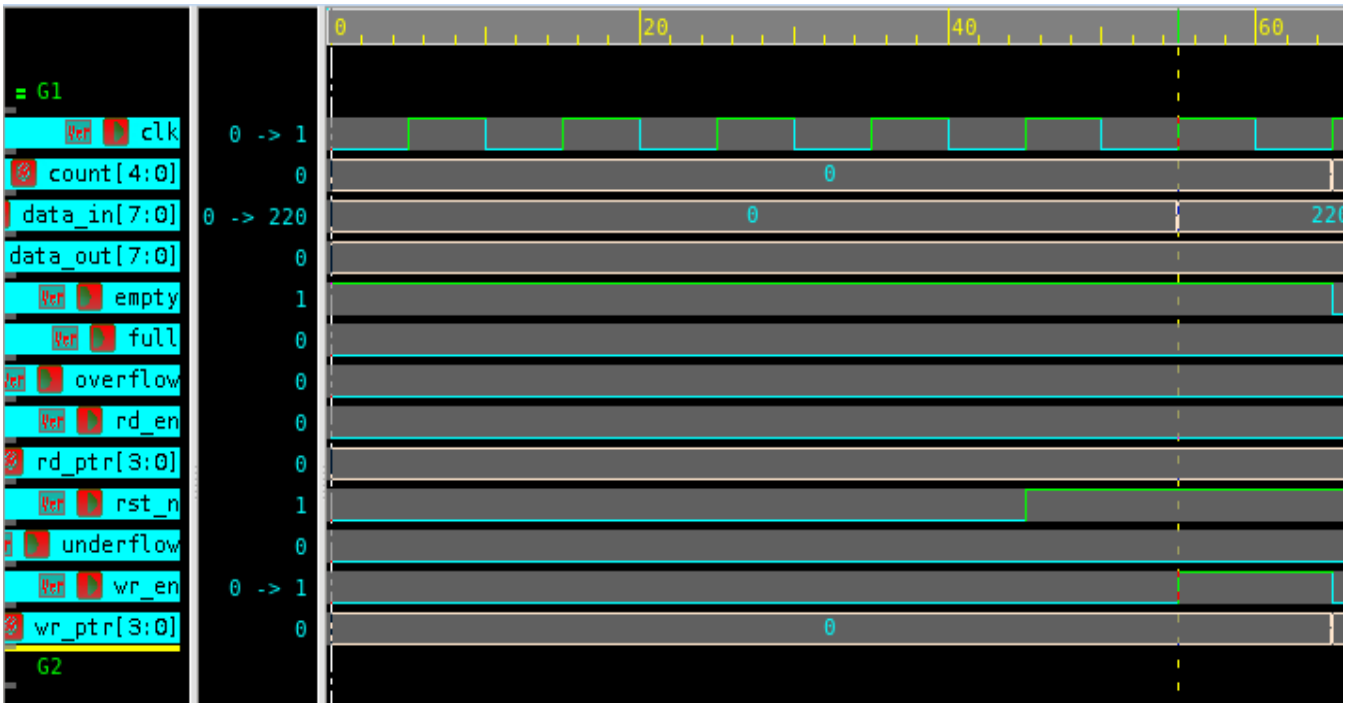
(Schematic View of 16x8 Synchronous fifo - by VCS and Verdi tool)



(This is the internal schematic view of synthesizable circuits of 16x8 synchronous FIFO)

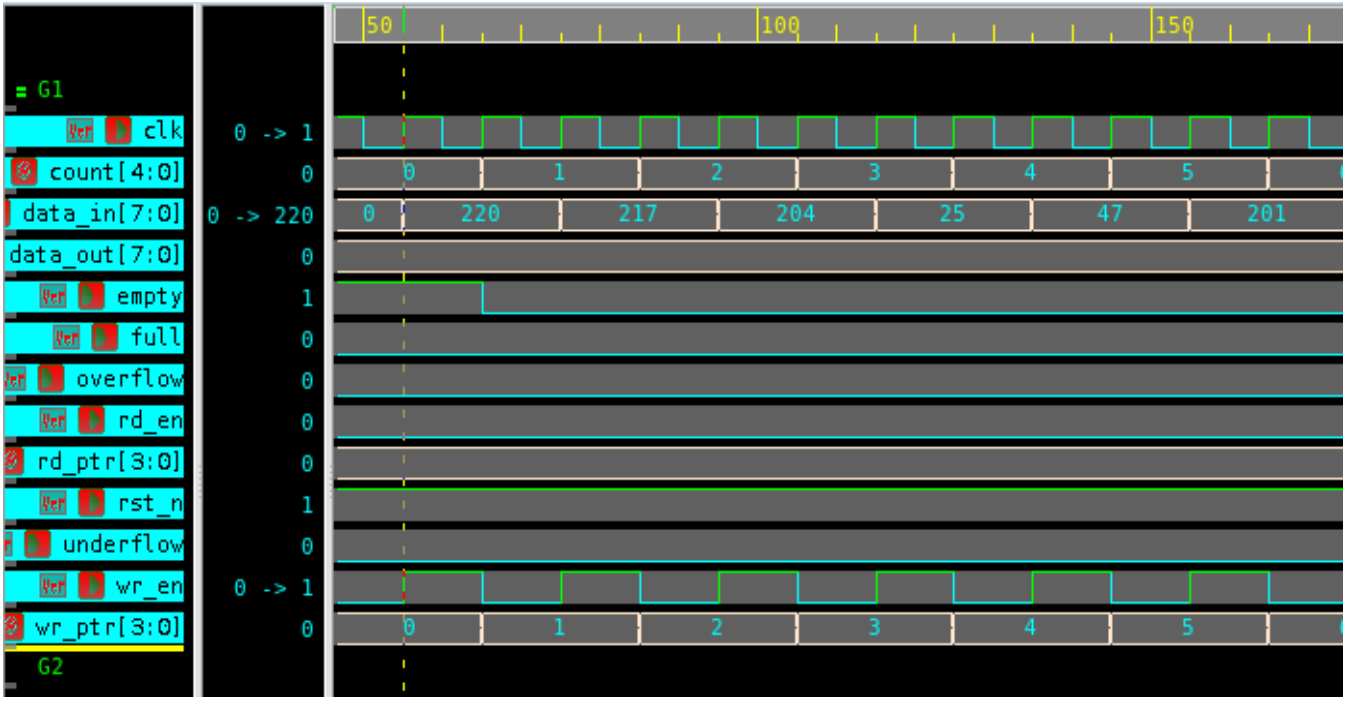
Waveform-

Test case1: Reset check

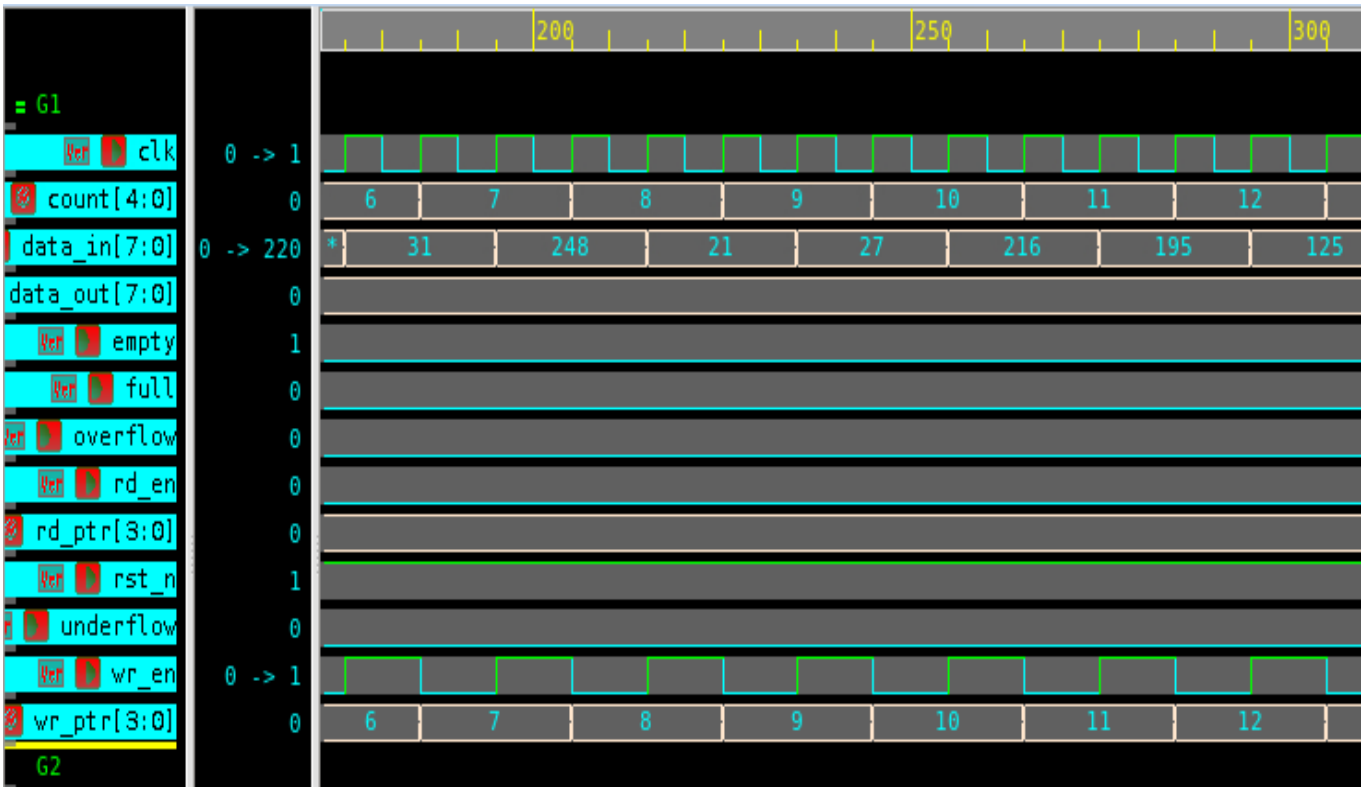


(Test status: checked pass)

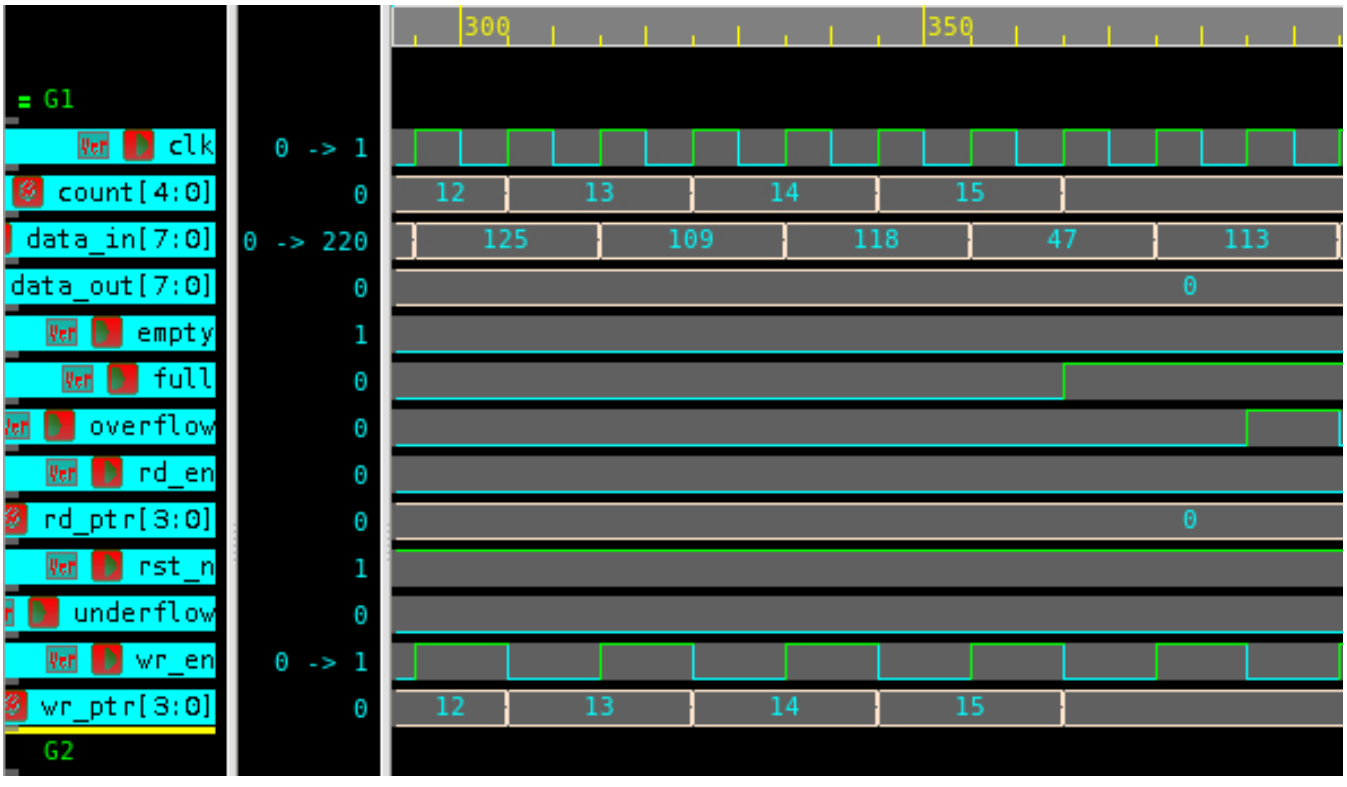
Test case2: Write data_in FIFO until full completely



(Test status: checked pass)

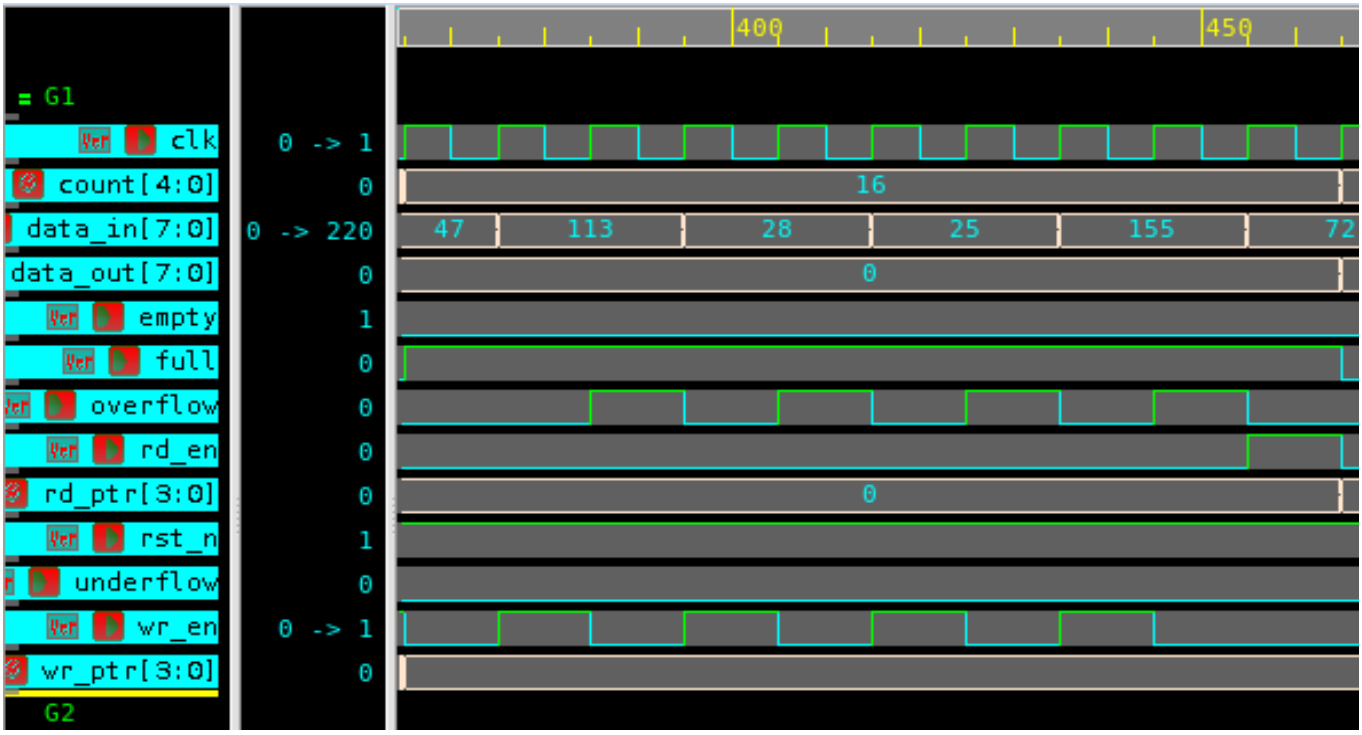


(Test status: checked pass)



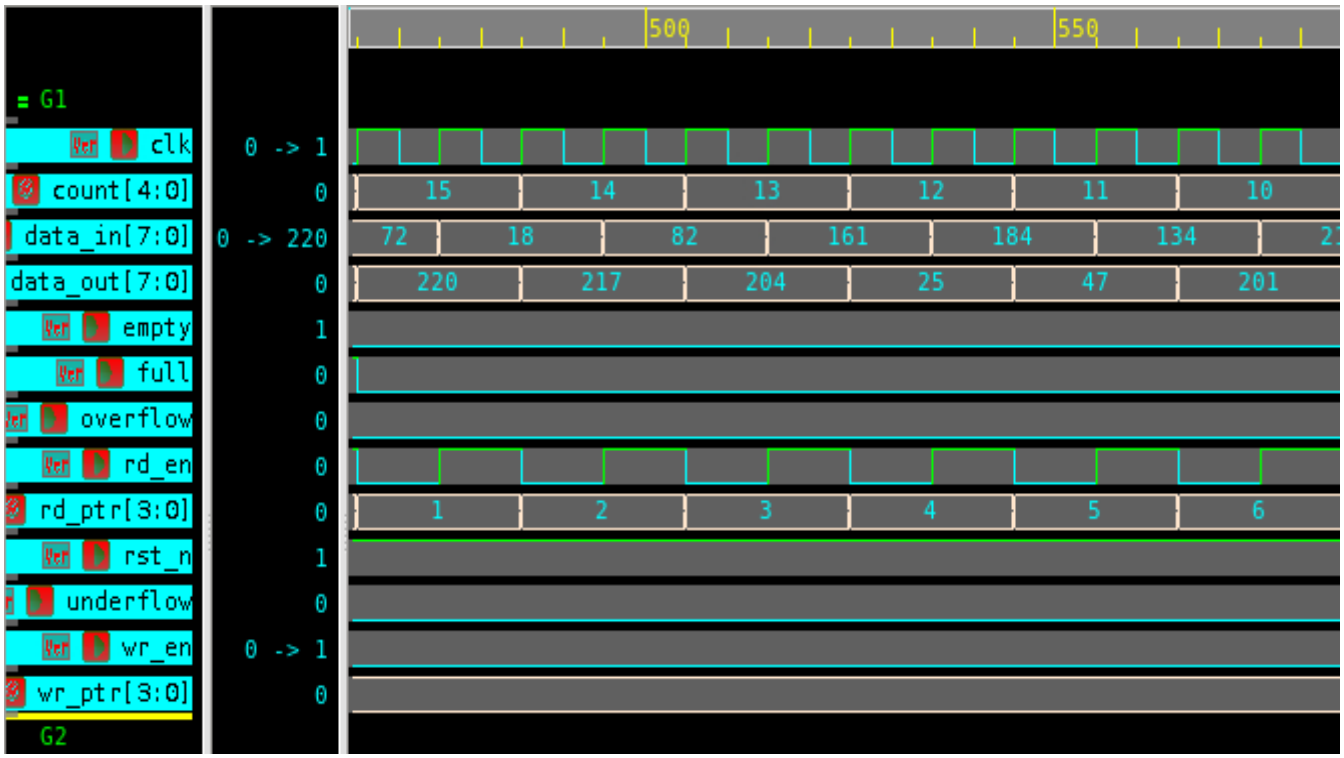
(Test status: checked pass)

Test case3: Try to write in full FIFO check (overflow)

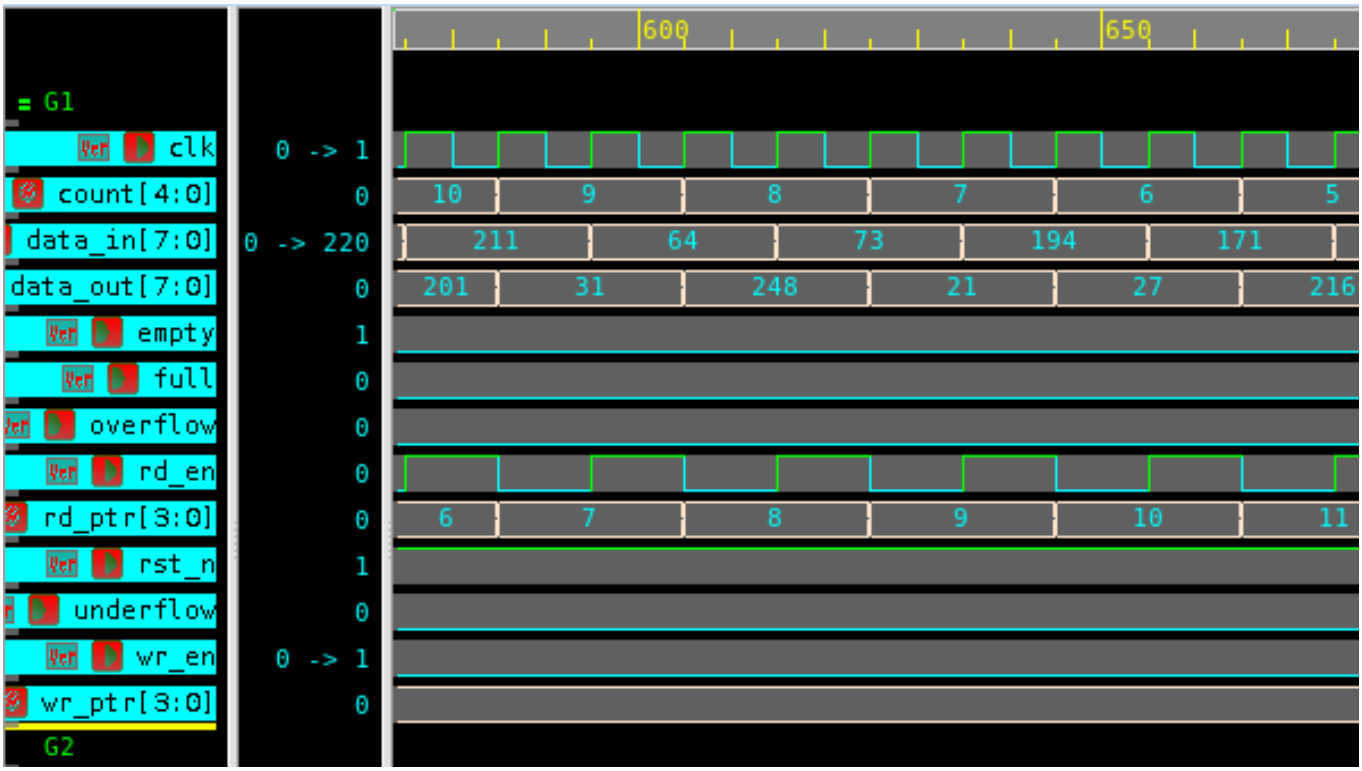


(Test status: checked pass)

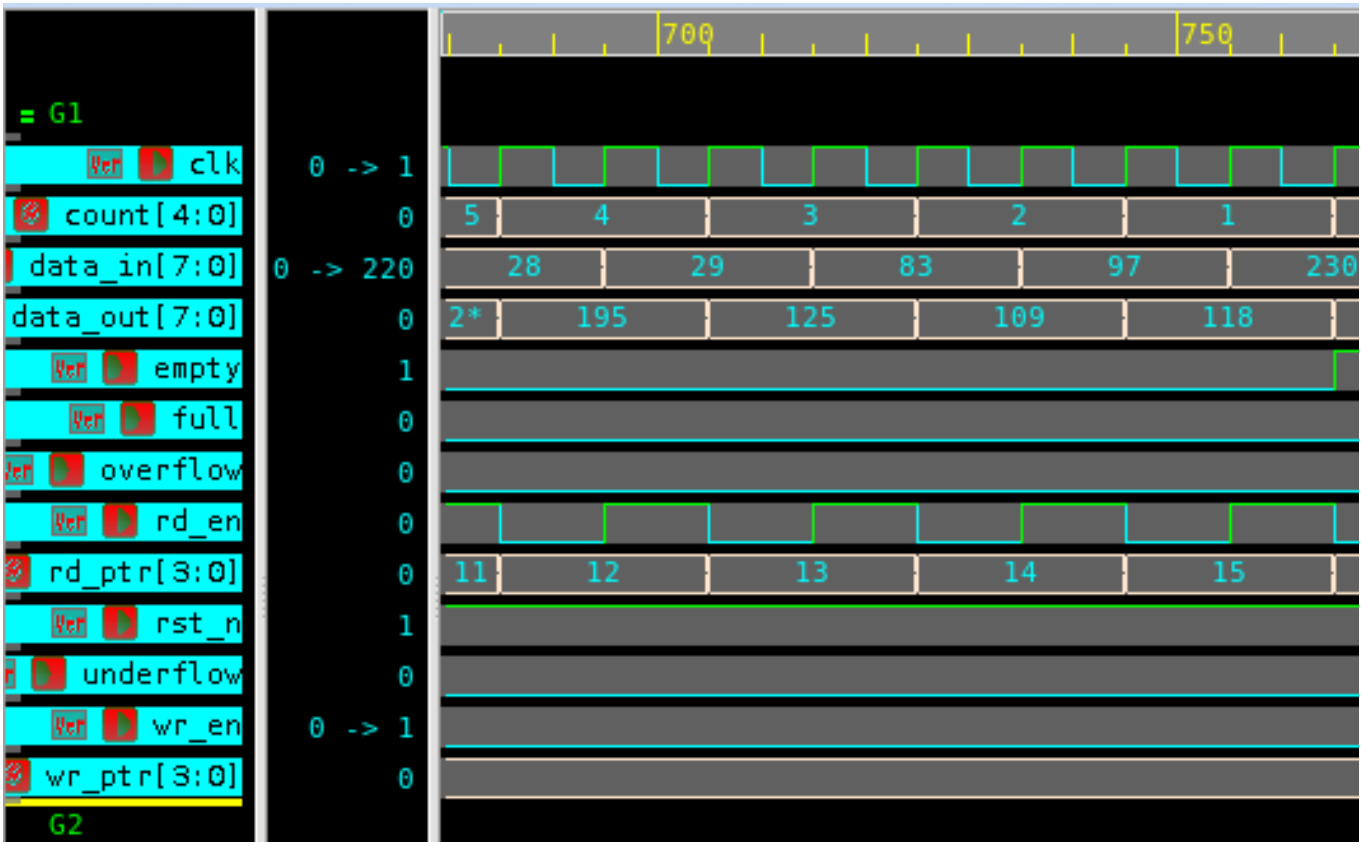
Test case4: Read all data until empty FIFO check



(Test status: checked pass)

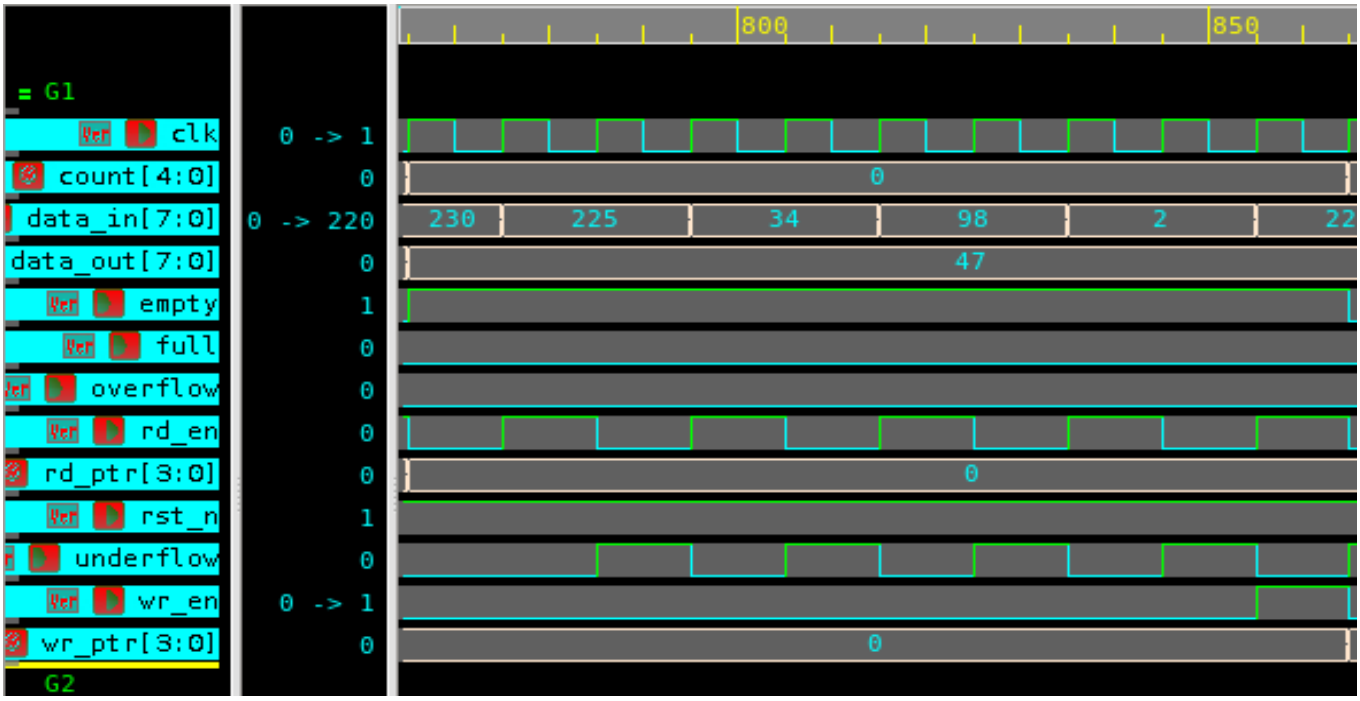


(Test status: checked pass)



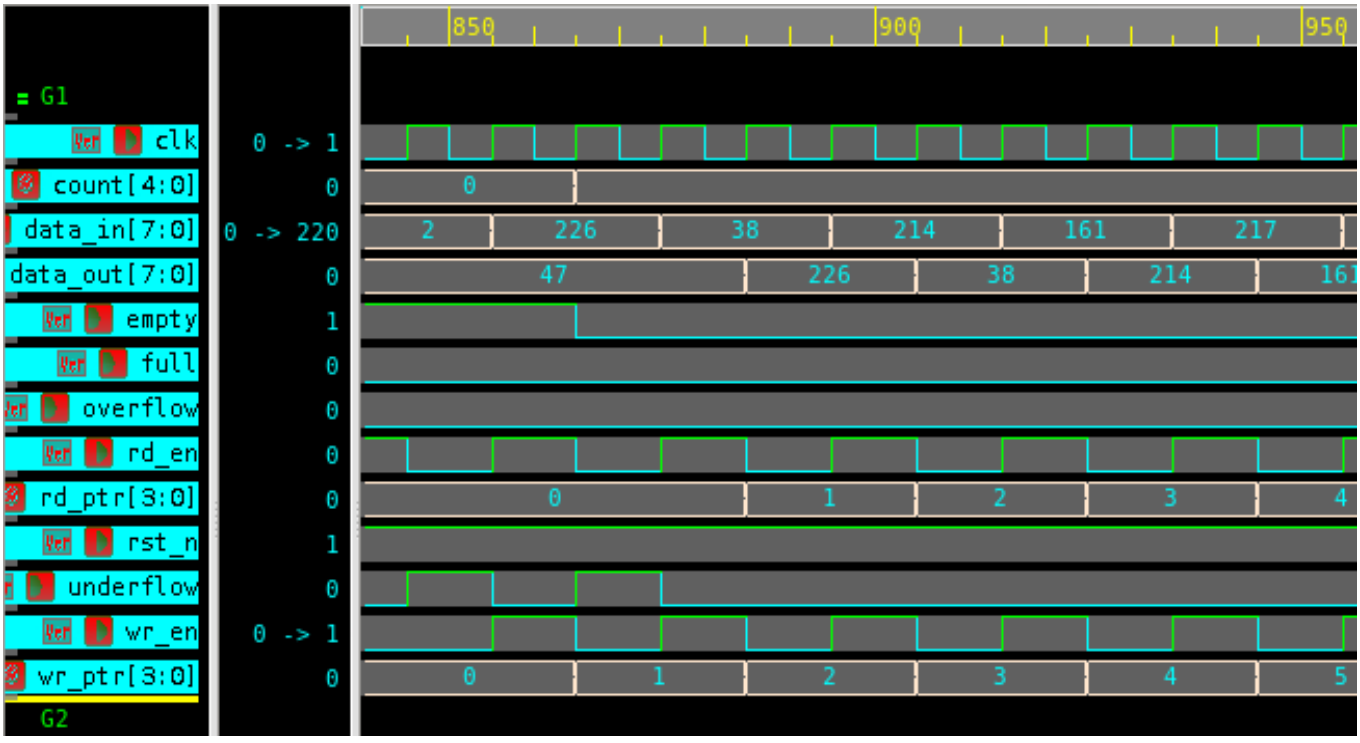
(Test status: checked pass)

Test case5: Try to read more data from empty FIFO (underflow) check-

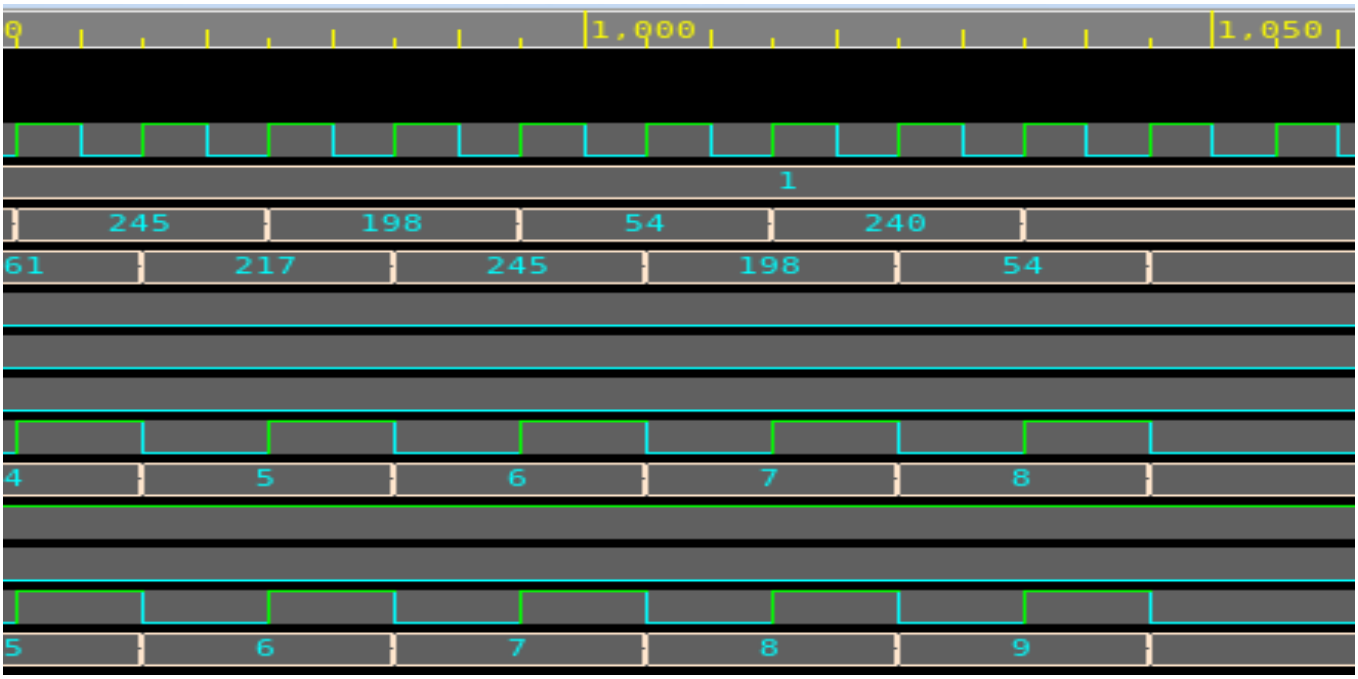


(Test status: checked pass)

Test case6: simultaneously write/read operation check-

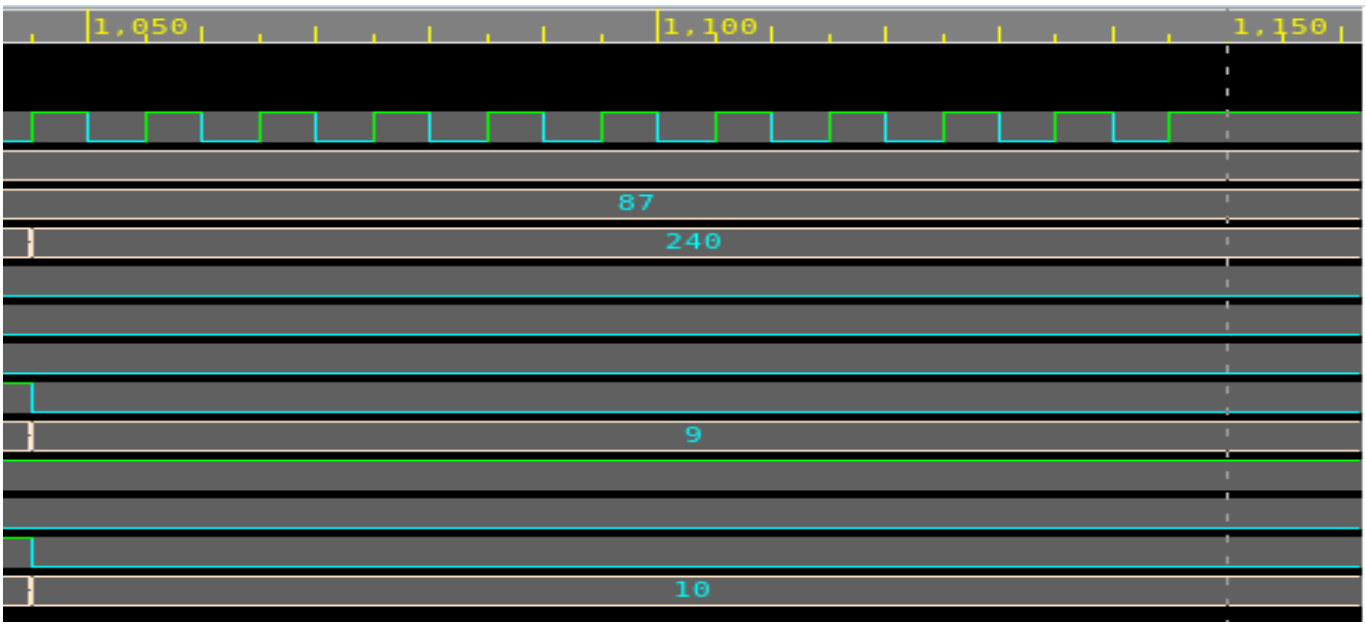


(Test status: checked pass)



(Test status: checked pass)

Test case7: No read/write operation (Idle case) check-



(Test status: checked pass)

Finally, our design is functionally correct and passes all simulation cases: Reset, write until full, Overflow detection, read until empty, underflow detection, simultaneous read/write operations, reuse memory addresses cyclically (wrap-around), Hold state, data correctness. **In digital design, FIFO is not just a buffer- it's a bridge that ensures seamless data flow.**

Coverage Report:-

Avg. Group Score:96.88% U+C:38 U:5 C:33 X:0
Avg. Group Inst. Score:96.88% U+C:38 U:5 C:33 X:0

Group	Score	Instances	U+C	U	C	X	Goal	Weight	AtLeast	PerInst	Overlap	AutoBinI	Missing
$unit::fifo_coverage::fifo...	96.88%		38	5	33	0	100%	1	1	0	1	64	64
cp_empty	100.00%		2	0	2	0	100%	1	1		1	0	
cp_full	100.00%		2	0	2	0	100%	1	1		1	0	
cp_op	100.00%		4	0	4	0	100%	1	1		1	0	
cp_overflow	100.00%		1	0	1	0	100%	1	1		1	0	
cp_rd_en	100.00%		2	0	2	0	100%	1	1		1	0	
cp_underflow	100.00%		1	0	1	0	100%	1	1		1	0	
cp_wr_en	100.00%		2	0	2	0	100%	1	1		1	0	
cross_op_state	68.75%		16	5	11	0	100%	1	1				0
cross_rd_empty	100.00%		4	0	4	0	100%	1	1				0
cross_wr_full	100.00%		4	0	4	0	100%	1	1				0

Message

The design 'simv.vdb' was loaded successfully.
The following test is loaded from "simv.vdb",
simv/test

(Test status: checked pass near 96.88%)